

DATABASE MANAGEMENT SYSTEMS

Course Project Report

Design and Implementation of a Database Management System for Courier Parcel Booking and Tracking Management System

Submitted by

Name: Keerthana Dara

Roll Number: 25WU0102125

Course: Data Base And Management System

Faculty Name: Kiran Mayee Adavala

Academic Year: 2026-27

Department : Ai&ML(Panthers)

2. Abstract

Courier companies handle thousands of parcels every day. Each parcel is booked at a branch, charged according to its weight and service type, moved through one or more hubs by vehicles, and finally handed to a recipient by a delivery agent. When this information is kept in paper registers or disconnected spreadsheets, parcels are hard to trace, charges are inconsistent and payment records are unreliable.

This project designs and implements a relational database for a Courier Parcel Booking and Tracking Management System. The design was captured in an Entity-Relationship (ER) diagram with 13 entities: Customer, Booking, Parcel, Service_Type, Branch, Hub, Delivery_Agent, Vehicle, Route, Scan_Event, Delivery_Attempt, Charge and Payment. The ER model was converted into a relational schema in Third Normal Form, with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints enforcing data integrity.

The database was created using SQL DDL statements, populated with sample data using DML statements, and queried with joins, aggregations, grouping and a view (Parcel_Tracking). A web-based user interface named "Parcel Ledger" was developed on top of the database. It lets an operator view, add, edit and delete records in every table, track a parcel by its tracking number, view predefined SQL reports and inspect the relational schema. The report documents the design, implementation, queries, user interface and testing of the system.

Keywords: DBMS, Courier Management, Parcel Tracking, ER Diagram, Normalization, SQL, Relational Database.

3. Introduction and Problem Statement

3.1 Introduction

A courier service moves documents and goods between a sender and a receiver, usually across cities. A typical shipment involves many actors and events: the customer who books it, the branch that accepts it, the hubs that sort and transfer it, the vehicles that carry it, the agents who deliver it, and the payments that settle its cost. Keeping all of these connected requires a well-designed database.

3.2 Problem statement

Manual or loosely organised record keeping in courier operations leads to the following problems:

- A parcel cannot be located quickly because its movement between branches and hubs is not recorded in one place.
- Charges are calculated inconsistently since the service type, weight and extra charges are not stored together.
- Payment status cannot be reliably matched to a booking.
- Failed delivery attempts and their reasons are not recorded, so re-delivery cannot be planned.
- Duplicate or inconsistent customer, branch and vehicle data accumulates over time.

The problem addressed by this project is therefore: to design and implement a relational database, with a simple user interface, that stores and connects all booking, parcel, routing, delivery, charge and payment information of a courier company and allows any parcel to be tracked from booking to delivery.

4. Objectives and Scope

4.1 Objectives

- To analyse the data requirements of a courier parcel booking and tracking system.
- To design an ER diagram showing entities, attributes, keys and relationships with cardinalities.
- To convert the ER diagram into a normalised relational schema (up to 3NF).
- To implement the schema in SQL with proper integrity constraints.
- To populate the database with realistic sample data and write meaningful SQL queries.
- To build a user interface supporting insert, delete and view operations, and parcel tracking.
- To test the system with valid and invalid inputs.

4.2 Scope

Within scope: customer and booking management; parcel registration; service types and charge calculation; payments; branch, hub, route and vehicle management; delivery agent management; scan events for tracking; delivery attempts and failure reasons; reports and tracking.

Out of scope: real-time GPS tracking, mobile applications, customer login and authentication, online payment gateway integration, and customs/international documentation.

5. Software and Hardware Requirements

5.1 Software requirements

Component	Details
Database management system	SQLite 3 (relational, file based, SQL standard)
Query language	SQL (DDL, DML, DQL)
User interface	Web application "Parcel Ledger" (HTML/CSS/JavaScript front end)
Hosting	Replit cloud hosting (courier-parcel-dbms--keerthanadara90.replit.app)
Design tool	ER diagram drawn as a diagram image (13 entities)
Version control	Git and GitHub
Operating system	Windows 11
Browser	Microsoft Edge / Google Chrome (latest)

5.2 Hardware requirements

Component	Minimum requirement
Processor	Dual-core 2 GHz or higher
Memory (RAM)	4 GB or more
Storage	500 MB free disk space
Display	1366 x 768 resolution or higher
Network	Internet connection for the hosted web interface

6. ER Diagram

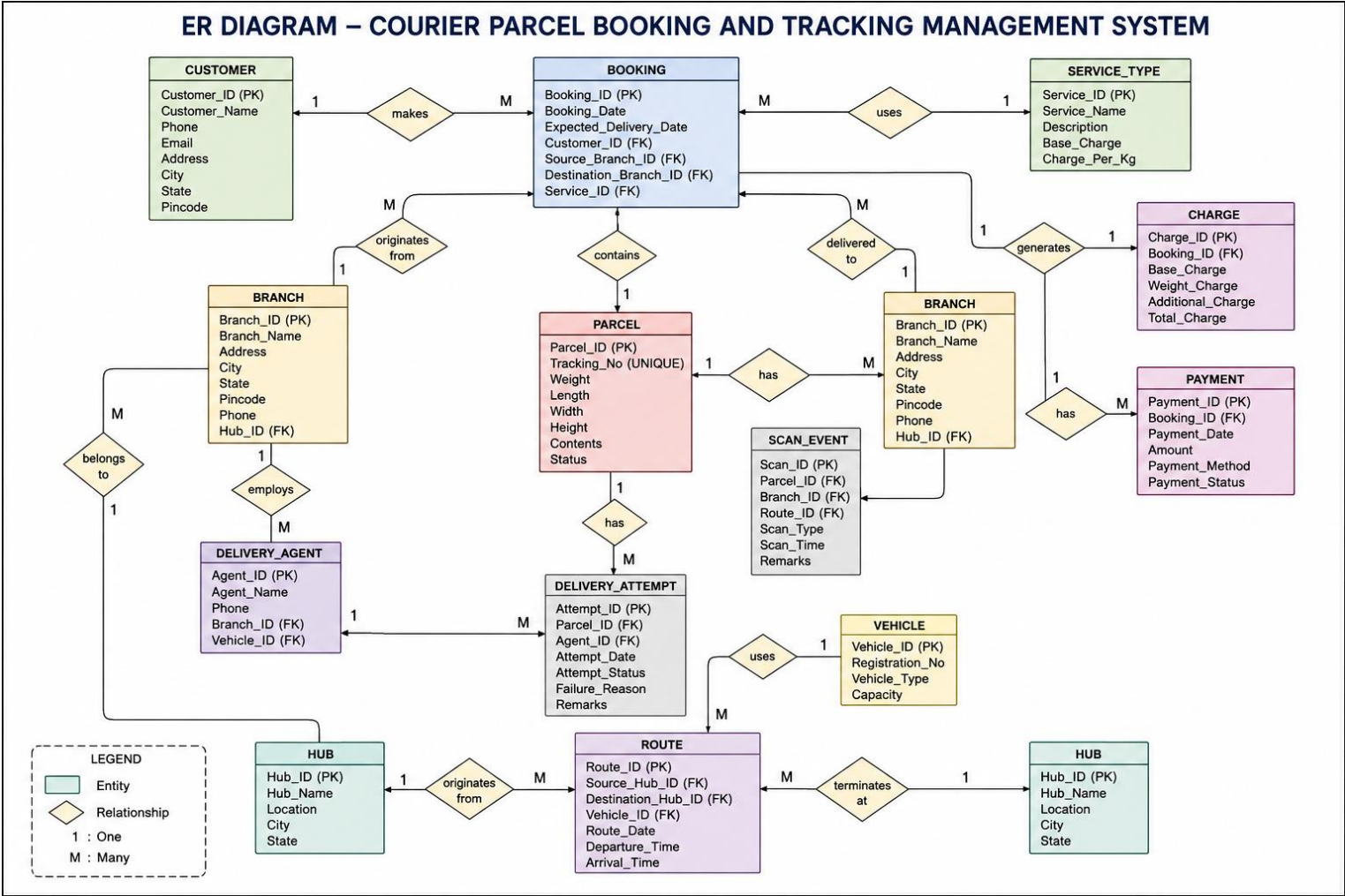


Figure 1: ER diagram of the Courier Parcel Booking and Tracking Management System

6.1 Entities

The ER diagram contains 13 entities. The Branch and Hub entities appear twice in the figure only to avoid crossing lines (source and destination); each is one table in the database.

Entity	Purpose	Primary key
Customer	Sender who books a parcel	Customer_ID
Booking	A booking transaction for one parcel	Booking_ID
Parcel	The physical shipment with tracking number	Parcel_ID
Service_Type	Courier service (standard, express, ...) and rates	Service_ID
Branch	Courier office where parcels are booked / delivered	Branch_ID
Hub	Sorting and transfer point; each branch belongs to a hub	Hub_ID
Delivery_Agent	Employee who delivers parcels	Agent_ID
Vehicle	Vehicle used by agents and on routes	Vehicle_ID
Route	A trip between two hubs by a vehicle	Route_ID
Scan_Event	Scan record created whenever a parcel moves or changes status	Scan_ID
Delivery_Attempt	Each attempt to deliver a parcel	Attempt_ID
Charge	Charge breakdown for a booking	Charge_ID
Payment	Payment made for a booking	Payment_ID

6.2 Relationships and cardinalities

Relationship	Entities	Cardinality
makes	Customer - Booking	1 : M
uses	Service_Type - Booking	1 : M
originates from	Branch - Booking (source branch)	1 : M
delivered to	Branch - Booking (destination branch)	1 : M
contains	Booking - Parcel	1 : 1
generates	Booking - Charge	1 : 1
has	Booking - Payment	1 : M
has	Parcel - Scan_Event	1 : M
has	Parcel - Delivery_Attempt	1 : M
employs	Branch - Delivery_Agent	1 : M
belongs to	Hub - Branch	1 : M
originates from / terminates at	Hub - Route (source and destination hub)	1 : M each
uses	Vehicle - Route	1 : M
assigned	Vehicle - Delivery_Agent	1 : M
makes	Delivery_Agent - Delivery_Attempt	1 : M

7. Relational Schema and Normalization

7.1 Relational schema

The ER diagram was mapped to the following 13 relations (PK = primary key, FK = foreign key):

Hub(hub_id (PK), hub_name, location, city, state)

Customer(customer_id (PK), customer_name, phone, email, address, city, state, pincode)

Branch(branch_id (PK), branch_name, address, city, state, pincode, phone, hub_id (FK))

Service_Type(service_id (PK), service_name, description, base_charge, charge_per_kg)

Vehicle(vehicle_id (PK), registration_no, vehicle_type, capacity)

Delivery_Agent(agent_id (PK), agent_name, phone, branch_id (FK), vehicle_id (FK))

Parcel(parcel_id (PK), tracking_no, weight, length, width, height, contents, status)

Booking(booking_id (PK), customer_id (FK), parcel_id (FK), service_id (FK), source_branch_id (FK), destination_branch_id (FK), booking_date, expected_delivery_date)

Route(route_id (PK), source_hub_id (FK), destination_hub_id (FK), vehicle_id (FK), route_date, departure_time, arrival_time)

Scan_Event(scan_id (PK), parcel_id (FK), branch_id (FK), route_id (FK), scan_type, scan_time, remarks)

Delivery_Attempt(attempt_id (PK), parcel_id (FK), agent_id (FK), attempt_date, attempt_status, failure_reason, remarks)

Charge(charge_id (PK), booking_id (FK), base_charge, weight_charge, additional_charge, total_charge)

Payment(payment_id (PK), booking_id (FK), payment_date, amount, payment_method, payment_status)

Mapping rules used: every strong entity became a relation with its key as primary key; every 1:M relationship was implemented by placing the primary key of the "one" side as a foreign key in the "many" side (for example Branch.hub_id, Booking.customer_id); the 1:1 relationships Booking-Parcel and Booking-Charge were implemented with a UNIQUE foreign key; Booking has two foreign keys to Branch (source_branch_id and destination_branch_id) and Route has two foreign keys to Hub (source_hub_id and destination_hub_id).

7.2 Normalization

First Normal Form (1NF). All attributes hold single, atomic values. Dimensions of a parcel are split into length, width and height; an address is split into address, city, state and pincode; there are no repeating groups or multi-valued columns.

Second Normal Form (2NF). Every relation has a single-column primary key, so no non-key attribute can depend on part of a key. All non-key attributes depend on the whole key.

Third Normal Form (3NF). No non-key attribute depends on another non-key attribute. Examples of how transitive dependencies were removed:

- Customer details are stored once in Customer, not repeated in every Booking.
- Service name and per-kg rate live in Service_Type; Booking only stores service_id.
- Branch city and address live in Branch; hub details live in Hub and are reached through Branch.hub_id.
- Vehicle details live in Vehicle; Delivery_Agent and Route only store vehicle_id.

- Charge details are separated from Booking, and payment details from Booking, so each fact is stored in exactly one place.

All 13 relations are in 3NF; every determinant is a candidate key, so the schema is also in Boyce-Codd Normal Form for the dependencies identified.

8. Data Dictionary

The following tables list the structure, data types and constraints of every table, as created in the database.

8.1 Hub

Column	Data type	Constraints	Description
hub_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique hub identifier
hub_name	TEXT	NOT NULL	Name of the sorting/transfer hub
location	TEXT	NOT NULL	Locality of the hub
city	TEXT	NOT NULL	City
state	TEXT	NOT NULL	State

8.2 Customer

Column	Data type	Constraints	Description
customer_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique customer identifier
customer_name	TEXT	NOT NULL	Full name of the sender
phone	TEXT	NOT NULL, UNIQUE	Contact number
email	TEXT	UNIQUE	E-mail address
address	TEXT	NOT NULL	Street address
city	TEXT	NOT NULL	City
state	TEXT	NOT NULL	State
pincode	TEXT	NOT NULL	Postal PIN code

8.3 Branch

Column	Data type	Constraints	Description
branch_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique branch identifier
branch_name	TEXT	NOT NULL	Branch name
address	TEXT	NOT NULL	Branch address
city	TEXT	NOT NULL	City
state	TEXT	NOT NULL	State
pincode	TEXT	NOT NULL	PIN code
phone	TEXT	NOT NULL, UNIQUE	Branch contact number
hub_id	INTEGER	NOT NULL, FK -> Hub(hub_id)	Hub to which the branch belongs

8.4 Service_Type

Column	Data type	Constraints	Description
service_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique service identifier
service_name	TEXT	NOT NULL, UNIQUE	Name of the courier service
description	TEXT	-	Short description of the service
base_charge	REAL	NOT NULL, CHECK(base_charge > 0)	Fixed starting charge (Rs.)
charge_per_kg	REAL	NOT NULL, CHECK(charge_per_kg > 0)	Charge per kilogram (Rs.)

8.5 Vehicle

Column	Data type	Constraints	Description
vehicle_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique vehicle identifier
registration_no	TEXT	NOT NULL, UNIQUE	Vehicle registration number
vehicle_type	TEXT	NOT NULL	Van, truck, bike, etc.
capacity	REAL	NOT NULL, CHECK(capacity > 0)	Load capacity (kg)

8.6 Delivery_Agent

Column	Data type	Constraints	Description
agent_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique agent identifier
agent_name	TEXT	NOT NULL	Agent name
phone	TEXT	NOT NULL, UNIQUE	Agent contact number
branch_id	INTEGER	NOT NULL, FK -> Branch(branch_id)	Branch the agent works for
vehicle_id	INTEGER	FK -> Vehicle(vehicle_id)	Vehicle assigned to the agent

8.7 Parcel

Column	Data type	Constraints	Description
parcel_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique parcel identifier
tracking_no	TEXT	NOT NULL, UNIQUE	Public tracking number
weight	REAL	NOT NULL, CHECK(weight > 0)	Weight (kg)
length	REAL	CHECK(length > 0)	Length (cm)
width	REAL	CHECK(width > 0)	Width (cm)
height	REAL	CHECK(height > 0)	Height (cm)
contents	TEXT	-	Description of contents
status	TEXT	NOT NULL, DEFAULT 'BOOKED'	Current life-cycle status

8.8 Booking

Column	Data type	Constraints	Description
booking_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique booking identifier
customer_id	INTEGER	NOT NULL, FK -> Customer(customer_id)	Customer who made the booking
parcel_id	INTEGER	NOT NULL, UNIQUE, FK -> Parcel(parcel_id)	Parcel booked (one booking per parcel)
service_id	INTEGER	NOT NULL, FK -> Service_Type(service_id)	Service type chosen
source_branch_id	INTEGER	NOT NULL, FK -> Branch(branch_id)	Branch where parcel is booked
destination_branch_id	INTEGER	NOT NULL, FK -> Branch(branch_id)	Branch where parcel is delivered
booking_date	TEXT	NOT NULL	Date of booking
expected_delivery_date	TEXT	NOT NULL	Promised delivery date

8.9 Route

Column	Data type	Constraints	Description
route_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique route identifier
source_hub_id	INTEGER	NOT NULL, FK -> Hub(hub_id)	Hub where the trip starts
destination_hub_id	INTEGER	NOT NULL, FK -> Hub(hub_id)	Hub where the trip ends
vehicle_id	INTEGER	NOT NULL, FK -> Vehicle(vehicle_id)	Vehicle used on the trip

Column	Data type	Constraints	Description
route_date	TEXT	NOT NULL	Date of the trip
departure_time	TEXT	-	Departure time
arrival_time	TEXT	-	Arrival time

8.10 Scan_Event

Column	Data type	Constraints	Description
scan_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique scan identifier
parcel_id	INTEGER	NOT NULL, FK -> Parcel(parcel_id)	Parcel scanned
branch_id	INTEGER	FK -> Branch(branch_id)	Branch where the scan happened
route_id	INTEGER	FK -> Route(route_id)	Route on which the parcel travelled
scan_type	TEXT	NOT NULL	Event type (BOOKED, IN_TRANSIT, ...)
scan_time	TEXT	NOT NULL	Date and time of the scan
remarks	TEXT	-	Additional notes

8.11 Delivery_Attempt

Column	Data type	Constraints	Description
attempt_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique attempt identifier
parcel_id	INTEGER	NOT NULL, FK -> Parcel(parcel_id)	Parcel being delivered
agent_id	INTEGER	NOT NULL, FK -> Delivery_Agent(agent_id)	Agent who attempted delivery
attempt_date	TEXT	NOT NULL	Date and time of the attempt
attempt_status	TEXT	NOT NULL	SUCCESS or FAILED
failure_reason	TEXT	-	Reason when the attempt fails
remarks	TEXT	-	Additional notes

8.12 Charge

Column	Data type	Constraints	Description
charge_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique charge identifier
booking_id	INTEGER	NOT NULL, UNIQUE, FK -> Booking(booking_id)	Booking being charged (one-to-one)
base_charge	REAL	NOT NULL, CHECK(base_charge > 0)	Base service charge (Rs.)
weight_charge	REAL	NOT NULL, CHECK(weight_charge >= 0)	Weight-based charge (Rs.)
additional_charge	REAL	CHECK(additional_charge >= 0), DEFAULT 0	Extra charges (Rs.)
total_charge	REAL	NOT NULL, CHECK(total_charge > 0)	Total payable (Rs.)

8.13 Payment

Column	Data type	Constraints	Description
payment_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique payment identifier
booking_id	INTEGER	NOT NULL, FK -> Booking(booking_id)	Booking being paid for
payment_date	TEXT	NOT NULL	Date of payment
amount	REAL	NOT NULL, CHECK(amount > 0)	Amount paid (Rs.)
payment_method	TEXT	NOT NULL	UPI, CARD or CASH
payment_status	TEXT	NOT NULL, DEFAULT 'SUCCESS'	Payment outcome

9. SQL Commands Used (DDL and DML) with Sample Outputs

9.1 DDL: table creation

The tables are created in an order that satisfies the foreign key dependencies (Hub and Customer first, Payment last). Foreign key enforcement is switched on with PRAGMA foreign_keys = ON.

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS Hub (
    hub_id INTEGER PRIMARY KEY AUTOINCREMENT,
    hub_name TEXT NOT NULL,
    location TEXT NOT NULL,
    city TEXT NOT NULL,
    state TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Customer (
    customer_id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_name TEXT NOT NULL,
    phone TEXT NOT NULL UNIQUE,
    email TEXT UNIQUE,
    address TEXT NOT NULL,
    city TEXT NOT NULL,
    state TEXT NOT NULL,
    pincode TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Branch (
    branch_id INTEGER PRIMARY KEY AUTOINCREMENT,
    branch_name TEXT NOT NULL,
    address TEXT NOT NULL,
    city TEXT NOT NULL,
    state TEXT NOT NULL,
    pincode TEXT NOT NULL,
    phone TEXT NOT NULL UNIQUE,
    hub_id INTEGER NOT NULL,
    FOREIGN KEY (hub_id) REFERENCES Hub(hub_id)
);

CREATE TABLE IF NOT EXISTS Service_Type (
    service_id INTEGER PRIMARY KEY AUTOINCREMENT,
    service_name TEXT NOT NULL UNIQUE,
    description TEXT,
    base_charge REAL NOT NULL CHECK(base_charge > 0),
    charge_per_kg REAL NOT NULL CHECK(charge_per_kg > 0)
);

CREATE TABLE IF NOT EXISTS Vehicle (
    vehicle_id INTEGER PRIMARY KEY AUTOINCREMENT,
    registration_no TEXT NOT NULL UNIQUE,
    vehicle_type TEXT NOT NULL,
    capacity REAL NOT NULL CHECK(capacity > 0)
);

CREATE TABLE IF NOT EXISTS Delivery_Agent (
    agent_id INTEGER PRIMARY KEY AUTOINCREMENT,
    agent_name TEXT NOT NULL,
    phone TEXT NOT NULL UNIQUE,
    branch_id INTEGER NOT NULL,
    vehicle_id INTEGER,
    FOREIGN KEY (branch_id) REFERENCES Branch(branch_id),
    FOREIGN KEY (vehicle_id) REFERENCES Vehicle(vehicle_id)
);

CREATE TABLE IF NOT EXISTS Parcel (
    parcel_id INTEGER PRIMARY KEY AUTOINCREMENT,
    tracking_no TEXT NOT NULL UNIQUE,
    weight REAL NOT NULL CHECK(weight > 0),
```

```

length REAL CHECK(length > 0),
width REAL CHECK(width > 0),
height REAL CHECK(height > 0),
contents TEXT,
status TEXT NOT NULL DEFAULT 'BOOKED'
);

CREATE TABLE IF NOT EXISTS Booking (
    booking_id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER NOT NULL,
    parcel_id INTEGER NOT NULL UNIQUE,
    service_id INTEGER NOT NULL,
    source_branch_id INTEGER NOT NULL,
    destination_branch_id INTEGER NOT NULL,
    booking_date TEXT NOT NULL,
    expected_delivery_date TEXT NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (parcel_id) REFERENCES Parcel(parcel_id),
    FOREIGN KEY (service_id) REFERENCES Service_Type(service_id),
    FOREIGN KEY (source_branch_id) REFERENCES Branch(branch_id),
    FOREIGN KEY (destination_branch_id) REFERENCES Branch(branch_id)
);

CREATE TABLE IF NOT EXISTS Route (
    route_id INTEGER PRIMARY KEY AUTOINCREMENT,
    source_hub_id INTEGER NOT NULL,
    destination_hub_id INTEGER NOT NULL,
    vehicle_id INTEGER NOT NULL,
    route_date TEXT NOT NULL,
    departure_time TEXT,
    arrival_time TEXT,
    FOREIGN KEY (source_hub_id) REFERENCES Hub(hub_id),
    FOREIGN KEY (destination_hub_id) REFERENCES Hub(hub_id),
    FOREIGN KEY (vehicle_id) REFERENCES Vehicle(vehicle_id)
);

CREATE TABLE IF NOT EXISTS Scan_Event (
    scan_id INTEGER PRIMARY KEY AUTOINCREMENT,
    parcel_id INTEGER NOT NULL,
    branch_id INTEGER,
    route_id INTEGER,
    scan_type TEXT NOT NULL,
    scan_time TEXT NOT NULL,
    remarks TEXT,
    FOREIGN KEY (parcel_id) REFERENCES Parcel(parcel_id),
    FOREIGN KEY (branch_id) REFERENCES Branch(branch_id),
    FOREIGN KEY (route_id) REFERENCES Route(route_id)
);

CREATE TABLE IF NOT EXISTS Delivery_Attempt (
    attempt_id INTEGER PRIMARY KEY AUTOINCREMENT,
    parcel_id INTEGER NOT NULL,
    agent_id INTEGER NOT NULL,
    attempt_date TEXT NOT NULL,
    attempt_status TEXT NOT NULL,
    failure_reason TEXT,
    remarks TEXT,
    FOREIGN KEY (parcel_id) REFERENCES Parcel(parcel_id),
    FOREIGN KEY (agent_id) REFERENCES Delivery_Agent(agent_id)
);

CREATE TABLE IF NOT EXISTS Charge (
    charge_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER NOT NULL UNIQUE,
    base_charge REAL NOT NULL CHECK(base_charge > 0),
    weight_charge REAL NOT NULL CHECK(weight_charge >= 0),
    additional_charge REAL DEFAULT 0 CHECK(additional_charge >= 0),
    total_charge REAL NOT NULL CHECK(total_charge > 0),
    FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
);

CREATE TABLE IF NOT EXISTS Payment (

```

```

    payment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER NOT NULL,
    payment_date TEXT NOT NULL,
    amount REAL NOT NULL CHECK(amount > 0),
    payment_method TEXT NOT NULL,
    payment_status TEXT NOT NULL DEFAULT 'SUCCESS',
    FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
);

CREATE VIEW IF NOT EXISTS Parcel_Tracking AS
SELECT p.tracking_no, p.status, s.scan_type, s.scan_time, b.branch_name, s.remarks
FROM Parcel p
JOIN Scan_Event s ON p.parcel_id = s.parcel_id
LEFT JOIN Branch b ON s.branch_id = b.branch_id;

```

9.2 DML: inserting sample data

Each table was populated with sample data (15 rows per table, 25 rows for Scan_Event). A few representative statements are shown; the full script is available in the GitHub repository.

```

INSERT INTO Hub (hub_id, hub_name, location, city, state)
VALUES (1, 'Hyderabad Central Hub', 'Madhapur', 'Hyderabad', 'Telangana'),
       (2, 'Vijayawada Hub', 'Benz Circle', 'Vijayawada', 'Andhra Pradesh');

INSERT INTO Customer (customer_id, customer_name, phone, email, address, city, state, pincode)
VALUES (1, 'Rahul Kumar', '9000000001', 'rahul@gmail.com', 'Banjara Hills',
        'Hyderabad', 'Telangana', '500034');

INSERT INTO Parcel (parcel_id, tracking_no, weight, length, width, height, contents, status)
VALUES (1, 'TRK10001', 2.5, 20, 15, 10, 'Books', 'DELIVERED');

INSERT INTO Booking (booking_id, customer_id, parcel_id, service_id, source_branch_id,
                    destination_branch_id, booking_date, expected_delivery_date)
VALUES (1, 1, 1, 1, 1, 2, '2026-09-10', '2026-09-13');

INSERT INTO Scan_Event (scan_id, parcel_id, branch_id, route_id, scan_type, scan_time, remarks)
VALUES (1, 1, 1, NULL, 'BOOKED', '2026-09-10 09:00:00', 'Parcel booked');

INSERT INTO Payment (payment_id, booking_id, payment_date, amount, payment_method, payment_status)
VALUES (1, 1, '2026-09-10', 100, 'UPI', 'SUCCESS');

```

9.3 DML: update and delete

```

-- change the status of a parcel after delivery
UPDATE Parcel SET status = 'DELIVERED' WHERE tracking_no = 'TRK10004';

-- delete a record that is not referenced by any other table
DELETE FROM Parcel WHERE tracking_no = 'TRK10099';

```

A DELETE on a record that is still referenced by another table (for example a branch that has delivery agents) is rejected by the foreign key constraint, which protects the integrity of the data (see Section 13).

9.4 View creation

```

CREATE VIEW IF NOT EXISTS Parcel_Tracking AS
SELECT p.tracking_no, p.status, s.scan_type, s.scan_time, b.branch_name, s.remarks
FROM Parcel p
JOIN Scan_Event s ON p.parcel_id = s.parcel_id
LEFT JOIN Branch b ON s.branch_id = b.branch_id;

```

9.5 Sample output: record count of every table

Table_Name	Records
Hub	15
Customer	15
Branch	15
Service_Type	15
Vehicle	15
Delivery_Agent	15
Parcel	15
Booking	15
Route	15
Scan_Event	25
Delivery_Attempt	15
Charge	15
Payment	15

10. Queries with Outputs

Q1. List all tables in the database

Displays every table created in the schema.

```
SELECT name FROM sqlite_master WHERE type = 'table' AND name NOT LIKE 'sqlite_%' ORDER BY name;
```

name
Booking
Branch
Charge
Customer
Delivery_Agent
Delivery_Attempt
Hub
Parcel
Payment
Route
Scan_Event
Service_Type
Vehicle

Output: 13 rows.

Q2. Record count of every table

Verifies that sample data was loaded into all 13 tables.

```
SELECT 'Hub' AS Table_Name, COUNT(*) AS Records FROM Hub
UNION ALL SELECT 'Customer', COUNT(*) FROM Customer
UNION ALL SELECT 'Branch', COUNT(*) FROM Branch
UNION ALL SELECT 'Service_Type', COUNT(*) FROM Service_Type
UNION ALL SELECT 'Vehicle', COUNT(*) FROM Vehicle
UNION ALL SELECT 'Delivery_Agent', COUNT(*) FROM Delivery_Agent
UNION ALL SELECT 'Parcel', COUNT(*) FROM Parcel
UNION ALL SELECT 'Booking', COUNT(*) FROM Booking
UNION ALL SELECT 'Route', COUNT(*) FROM Route
UNION ALL SELECT 'Scan_Event', COUNT(*) FROM Scan_Event
UNION ALL SELECT 'Delivery_Attempt', COUNT(*) FROM Delivery_Attempt
UNION ALL SELECT 'Charge', COUNT(*) FROM Charge
UNION ALL SELECT 'Payment', COUNT(*) FROM Payment;
```

Table_Name	Records
Hub	15
Customer	15
Branch	15
Service_Type	15
Vehicle	15
Delivery_Agent	15
Parcel	15
Booking	15
Route	15

Table_Name	Records
Scan_Event	25
Delivery_Attempt	15
Charge	15
Payment	15

Output: 13 rows.

Q3. Complete booking information

Joins Booking, Customer, Parcel and Service_Type to show who booked which parcel under which service.

```
SELECT b.booking_id, c.customer_name, p.tracking_no, s.service_name, p.weight, p.status
FROM Booking b
JOIN Customer c ON b.customer_id = c.customer_id
JOIN Parcel p ON b.parcel_id = p.parcel_id
JOIN Service_Type s ON b.service_id = s.service_id;
```

booking_id	customer_name	tracking_no	service_name	weight	status
1	Rahul Kumar	TRK10001	Standard Delivery	2.5	DELIVERED
2	Priya Sharma	TRK10002	Express Delivery	1.2	IN TRANSIT
3	Arjun Reddy	TRK10003	Same Day Delivery	5	DELIVERED
4	Sneha Rao	TRK10004	Next Day Delivery	3.5	OUT FOR DELIVERY
5	Kiran Patel	TRK10005	Economy Delivery	7	IN TRANSIT
6	Ananya Singh	TRK10006	Premium Delivery	0.8	DELIVERED
7	Rohit Verma	TRK10007	Document Delivery	4.2	OUT FOR DELIVERY
8	Meena Iyer	TRK10008	Fragile Delivery	2	DELIVERED
9	Vivek Das	TRK10009	International Delivery	8.5	IN TRANSIT
10	Neha Nair	TRK10010	Bulk Delivery	1.5	OUT FOR DELIVERY
11	Aakash Shah	TRK10011	Business Delivery	6	DELIVERED
12	Pooja Mehta	TRK10012	Priority Delivery	3	IN TRANSIT
13	Varun Gupta	TRK10013	Doorstep Delivery	9	OUT FOR DELIVERY
14	Divya Joshi	TRK10014	Scheduled Delivery	1	BOOKED
15	Sanjay Kumar	TRK10015	Return Delivery	4.5	DELIVERED

Output: 15 rows.

Q4. Track a parcel (full scan history)

Returns the movement history of parcel TRK10001 in chronological order.

```
SELECT p.tracking_no, p.status, s.scan_type, s.scan_time, b.branch_name, s.remarks
FROM Parcel p
JOIN Scan_Event s ON p.parcel_id = s.parcel_id
LEFT JOIN Branch b ON s.branch_id = b.branch_id
WHERE p.tracking_no = 'TRK10001'
ORDER BY s.scan_time;
```

tracking_no	status	scan_type	scan_time	branch_name	remarks
TRK10001	DELIVERED	BOOKED	2026-09-10 09:00:00	Hyderabad Main Branch	Parcel booked
TRK10001	DELIVERED	PICKED_UP	2026-09-10 11:00:00	Hyderabad Main Branch	Parcel picked up
TRK10001	DELIVERED	ARRIVED	2026-09-12 14:00:00	Vijayawada Main Branch	Arrived at destination branch

tracking_no	status	scan_type	scan_time	branch_name	remarks
TRK10001	DELIVERED	OUT_FOR_DELIVERY	2026-09-13 09:00:00	Vijayawada Main Branch	Out for delivery
TRK10001	DELIVERED	DELIVERED	2026-09-13 16:00:00	Vijayawada Main Branch	Delivered successfully

Output: 5 rows.

Q5. Current location / latest status of a parcel

Returns only the most recent scan of parcel TRK10001.

```
SELECT p.tracking_no, p.status, b.branch_name, s.scan_type, s.scan_time
FROM Parcel p
JOIN Scan_Event s ON p.parcel_id = s.parcel_id
LEFT JOIN Branch b ON s.branch_id = b.branch_id
WHERE p.tracking_no = 'TRK10001'
ORDER BY s.scan_time DESC
LIMIT 1;
```

tracking_no	status	branch_name	scan_type	scan_time
TRK10001	DELIVERED	Vijayawada Main Branch	DELIVERED	2026-09-13 16:00:00

Output: 1 row.

Q6. Delivery agent and vehicle for a parcel

Shows which agent and vehicle handled the delivery attempts of TRK10001.

```
SELECT p.tracking_no, p.status, da.agent_name, da.phone AS agent_phone,
       v.registration_no, v.vehicle_type, d.attempt_date, d.attempt_status, d.failure_reason
FROM Parcel p
LEFT JOIN Delivery_Attempt d ON p.parcel_id = d.parcel_id
LEFT JOIN Delivery_Agent da ON d.agent_id = da.agent_id
LEFT JOIN Vehicle v ON da.vehicle_id = v.vehicle_id
WHERE p.tracking_no = 'TRK10001'
ORDER BY d.attempt_date DESC;
```

tracking_no	status	agent_name	agent_phone	registration_no	vehicle_type	attempt_date	attempt_status	failure_reason
TRK10001	DELIVERED	Suresh Reddy	9200000003	TN03EF1003	Van	2026-09-13 10:00:00	SUCCESS	NULL

Output: 1 row.

Q7. Total revenue from successful payments

Aggregate query using SUM.

```
SELECT SUM(amount) AS total_revenue FROM Payment WHERE payment_status = 'SUCCESS';
```

total_revenue
5545.5

Output: 1 row.

Q8. Parcel count by status

GROUP BY query summarising the parcel life-cycle.

```
SELECT status, COUNT(*) AS total_parcel FROM Parcel GROUP BY status;
```

status	total_parcel
BOOKED	1

status	total_parcels
DELIVERED	6
IN TRANSIT	4
OUT FOR DELIVERY	4

Output: 4 rows.

Q9. Failed delivery attempts

Lists failed attempts with the responsible agent and the failure reason.

```
SELECT p.tracking_no, da.agent_name, d.attempt_date, d.failure_reason
FROM Delivery_Attempt d
JOIN Parcel p ON d.parcel_id = p.parcel_id
JOIN Delivery_Agent da ON d.agent_id = da.agent_id
WHERE d.attempt_status = 'FAILED';
```

tracking_no	agent_name	attempt_date	failure_reason
TRK10002	Vikram Singh	2026-09-14 15:00:00	Customer unavailable
TRK10004	Amit Verma	2026-09-13 14:00:00	Wrong address
TRK10004	Amit Verma	2026-09-14 11:00:00	Customer unavailable
TRK10005	Vikram Singh	2026-09-16 16:00:00	Customer unavailable
TRK10007	Naveen Kumar	2026-09-17 14:00:00	Customer refused
TRK10009	Arun Nair	2026-09-18 12:00:00	Address inaccessible
TRK10010	Amit Verma	2026-09-19 13:00:00	Customer unavailable
TRK10012	Ramesh Meena	2026-09-20 16:00:00	Wrong address
TRK10013	Aman Gupta	2026-09-22 10:30:00	Customer unavailable

Output: 9 rows.

Q10. Branch-wise booking count

LEFT JOIN with GROUP BY so that branches without bookings are also listed.

```
SELECT br.branch_name, COUNT(b.booking_id) AS total_bookings
FROM Branch br
LEFT JOIN Booking b ON br.branch_id = b.source_branch_id
GROUP BY br.branch_id, br.branch_name;
```

branch_name	total_bookings
Hyderabad Main Branch	2
Vijayawada Main Branch	1
Chennai Main Branch	1
Warangal Main Branch	1
Bengaluru Main Branch	0
Mumbai Main Branch	1
Pune Main Branch	1
Delhi Main Branch	1
Kolkata Main Branch	1
Kochi Main Branch	1
Ahmedabad Main Branch	1
Jaipur Main Branch	1
Lucknow Main Branch	1

branch_name	total_bookings
Bhopal Main Branch	1
Coimbatore Main Branch	1

Output: 15 rows.

Q11. Customers from Hyderabad

Simple selection with a WHERE condition.

```
SELECT customer_id, customer_name, phone, city FROM Customer WHERE city = 'Hyderabad';
```

customer_id	customer_name	phone	city
1	Rahul Kumar	9000000001	Hyderabad
3	Arjun Reddy	9000000003	Hyderabad

Output: 2 rows.

Q12. Parcels heavier than 5 kg

Selection with ORDER BY.

```
SELECT tracking_no, weight, contents, status
FROM Parcel
WHERE weight > 5
ORDER BY weight DESC;
```

tracking_no	weight	contents	status
TRK10013	9	Furniture Parts	OUT FOR DELIVERY
TRK10009	8.5	Machine Parts	IN TRANSIT
TRK10005	7	Kitchen Items	IN TRANSIT
TRK10011	6	Clothes	DELIVERED

Output: 4 rows.

Q13. Payment details with customer and parcel

Four-table join across Payment, Booking, Customer and Parcel.

```
SELECT pay.payment_id, c.customer_name, p.tracking_no, pay.amount, pay.payment_method,
pay.payment_status
FROM Payment pay
JOIN Booking b ON pay.booking_id = b.booking_id
JOIN Customer c ON b.customer_id = c.customer_id
JOIN Parcel p ON b.parcel_id = p.parcel_id;
```

payment_id	customer_name	tracking_no	amount	payment_method	payment_status
1	Rahul Kumar	TRK10001	100	UPI	SUCCESS
2	Priya Sharma	TRK10002	156	CARD	SUCCESS
3	Arjun Reddy	TRK10003	375	UPI	SUCCESS
4	Sneha Rao	TRK10004	262.5	CASH	SUCCESS
5	Kiran Patel	TRK10005	155	UPI	SUCCESS
6	Ananya Singh	TRK10006	290	CARD	SUCCESS
7	Rohit Verma	TRK10007	112	UPI	SUCCESS
8	Meena Iyer	TRK10008	185	CASH	SUCCESS
9	Vivek Das	TRK10009	1450	CARD	SUCCESS
10	Neha Nair	TRK10010	840	UPI	SUCCESS

payment_id	customer_name	tracking_no	amount	payment_method	payment_status
11	Aakash Shah	TRK10011	470	CARD	SUCCESS
12	Pooja Mehta	TRK10012	455	UPI	SUCCESS
13	Varun Gupta	TRK10013	340	CASH	SUCCESS
14	Divya Joshi	TRK10014	170	UPI	SUCCESS
15	Sanjay Kumar	TRK10015	185	CARD	SUCCESS

Output: 15 rows.

Q14. Parcel_Tracking view

Queries the Parcel_Tracking view created over Parcel, Scan_Event and Branch.

```
SELECT * FROM Parcel_Tracking
WHERE tracking_no = 'TRK10002'
ORDER BY scan_time;
```

tracking_no	status	scan_type	scan_time	branch_name	remarks
TRK10002	IN TRANSIT	BOOKED	2026-09-11 10:00:00	Vijayawada Main Branch	Parcel booked
TRK10002	IN TRANSIT	IN_TRANSIT	2026-09-12 08:00:00	Vijayawada Main Branch	Parcel in transit
TRK10002	IN TRANSIT	ARRIVED	2026-09-13 15:00:00	Chennai Main Branch	Arrived at Chennai branch

Output: 3 rows.

Presentation-II query

To display all details of varun

```
SELECT * FROM Customer WHERE customer_name = 'Varun Gupta';
```

```
SELECT
    c.customer_id,
    c.customer_name,
    c.phone,
    c.email,
    c.address,
    c.city,
    c.state,
    c.pincode,
    b.booking_id,
    p.tracking_no,
    p.weight,
    p.contents,
    p.status,
```

```
s.service_name,  
b.booking_date,  
b.expected_delivery_date,  
sb.branch_name AS source_branch,  
db.branch_name AS destination_branch  
FROM Customer c  
JOIN Booking b ON c.customer_id = b.customer_id  
JOIN Parcel p ON b.parcel_id = p.parcel_id  
JOIN Service_Type s ON b.service_id = s.service_id  
JOIN Branch sb ON b.source_branch_id = sb.branch_id  
JOIN Branch db ON b.destination_branch_id = db.branch_id  
WHERE c.customer_name = 'Varun Gupta';
```

11. UI Design and Screenshots

11.1 Design overview

The user interface, "Parcel Ledger - Courier Operations", is a web application with a dark navigation sidebar and a light content area. It is hosted at <https://courier-parcel-dbms--keerthanadara90.replit.app>. The sidebar provides four operations pages and one page for each of the 13 relational tables.

Page	Purpose
Overview	Dashboard with record counts, recent bookings, parcel status ledger and latest scan events
Track a parcel	Search by tracking number and view the shipment record with scan history
Reports	18 predefined read-only SQL reports with the query text and results
ER schema	Relational model showing every table, its keys and fields
Table pages (13)	View all records of a table, search, add (insert), edit and delete records

Every table page shows the number of records, a search box, an "Add record" button for insert, and Edit and Delete buttons in the Actions column. Foreign keys are displayed as ID numbers that connect the record to other tables.

11.2 Overview dashboard

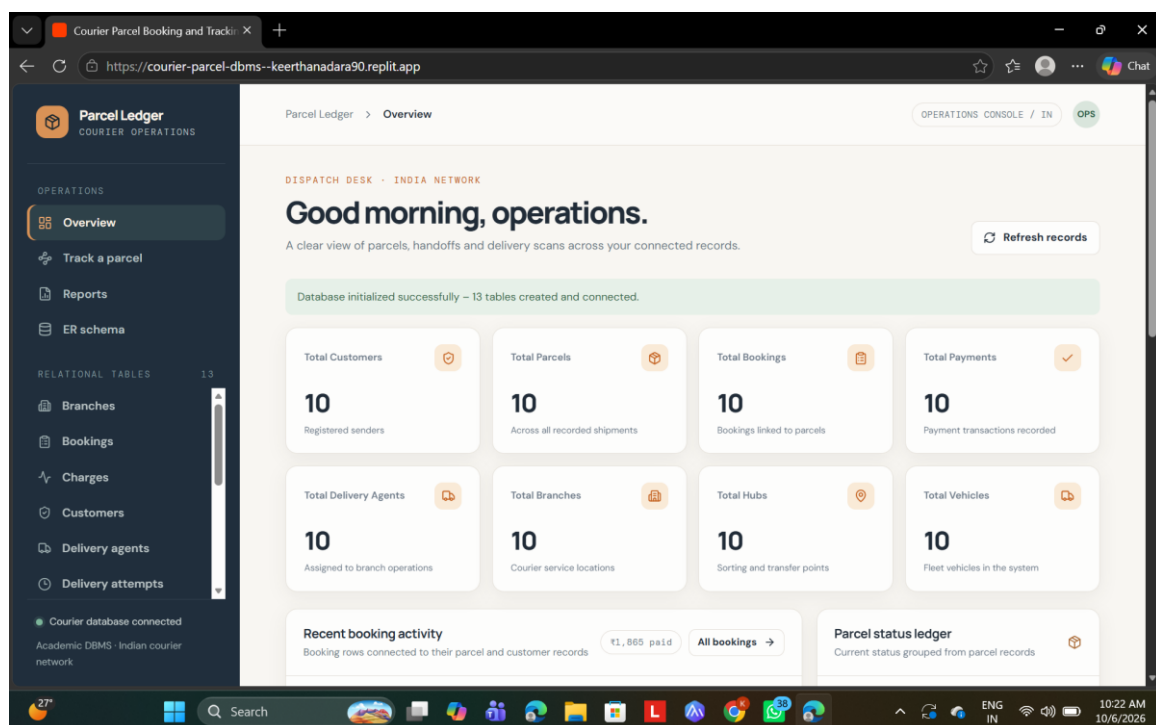


Figure 2: Overview dashboard showing totals for customers, parcels, bookings, payments, agents, branches, hubs and vehicles

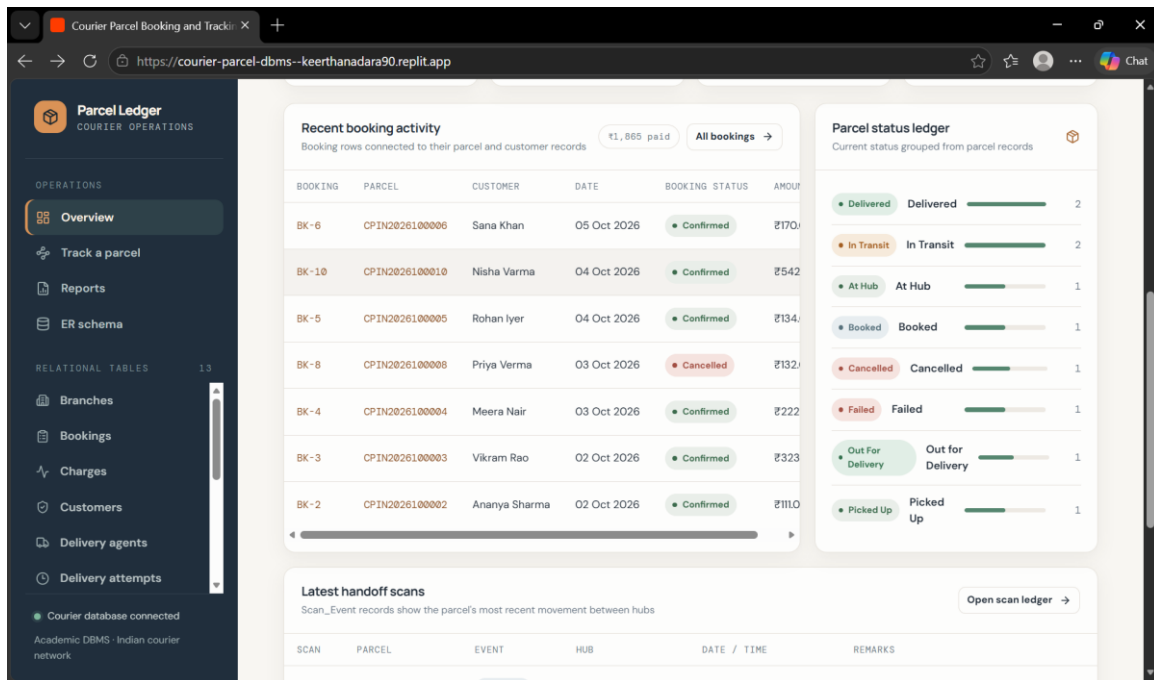


Figure 3: Recent booking activity, parcel status ledger and latest handoff scans

11.3 Track a parcel

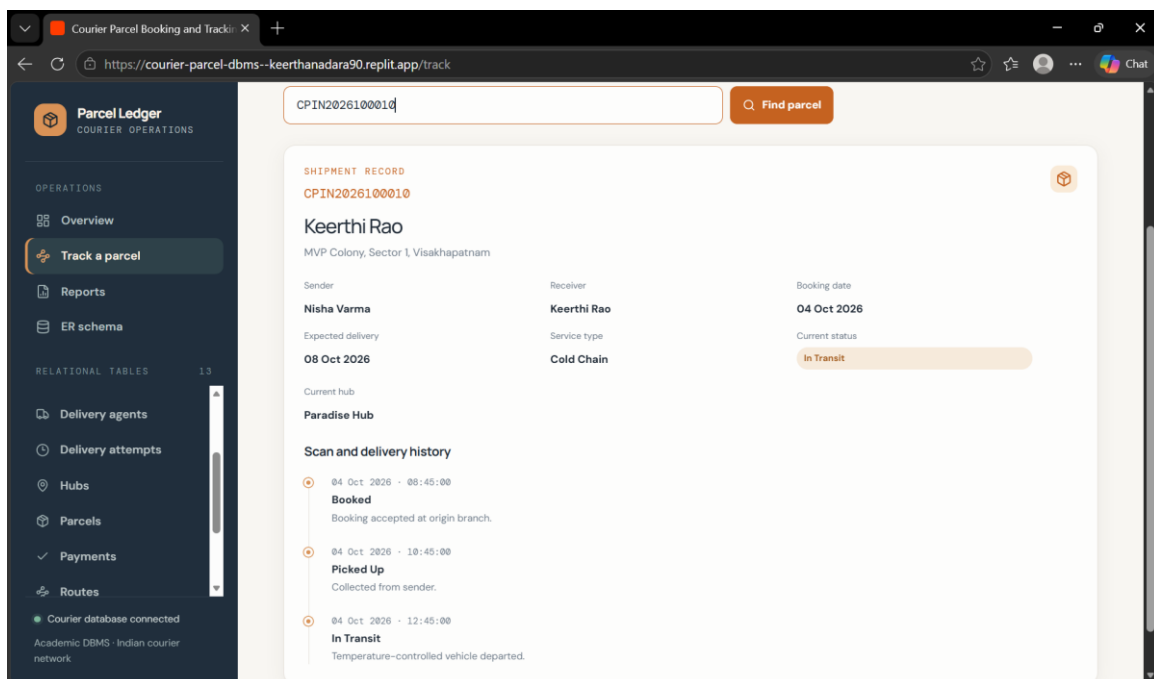


Figure 4: Tracking page: shipment record, current hub and scan and delivery history for a parcel

11.4 Reports and ER schema

Parcel Ledger
COURIER OPERATIONS

OPERATIONS

- Overview
- Track a parcel
- Reports**
- ER schema

RELATIONAL TABLES 13

- Branches
- Bookings
- Charges
- Customers
- Delivery agents
- Delivery attempts

Courier database connected
Academic DBMS - Indian courier network.

PREDEFINED SQL · READ-ONLY INSIGHTS

Reports from the records.

Choose from the system's predefined report library. Results are generated by the courier API against connected tables.

18 PREDEFINED REPORTS

Report library
18 queries available

- All customers**
List customer contact details and service locations.
- All parcels**
Review parcel records alongside service and customer names.
- All bookings**
Show booking records ordered from newest to oldest.
- Parcels in transit**
Find parcels currently moving between hubs.
- Delivered parcels**
Review successful deliveries and their receivers.
- Bookings with customer names**
Demonstrate joins across bookings, customers, parcels.

QUERY ALL-CUSTOMERS
All customers
List customer contact details and service locations.

Run again

```
SELECT customer_id, customer_name, phone, email, city
FROM customer
ORDER BY customer_name;
```

Query result

CUSTOMER ID	CUSTOMER NAME	PHONE	EMAIL
1	Aarav Reddy	9100001001	aarav.reddy@example.in
2	Ananya Sharma	9100001002	ananya.sharma@example.in
3	Vikram Rao	9100001003	vikram.rao@example.in
4	Meera Nair	9100001004	meera.nair@example.in
5	Rohan Iyer	9100001005	rohan.iyer@example.in

Figure 5: Reports page with the predefined query library, the SQL text and the query result

Parcel Ledger
COURIER OPERATIONS

OPERATIONS

- Overview
- Track a parcel
- Reports
- ER schema**

RELATIONAL TABLES 13

- Branches
- Bookings
- Charges
- Customers
- Delivery agents
- Delivery attempts

Courier database connected
Academic DBMS - Indian courier network.

RELATIONAL MODEL · 13 ENTITIES

One connected courier system.

A record map of customers, bookings, parcels, hubs and delivery scans. Primary keys and foreign keys reveal how an operational event joins back to its source.

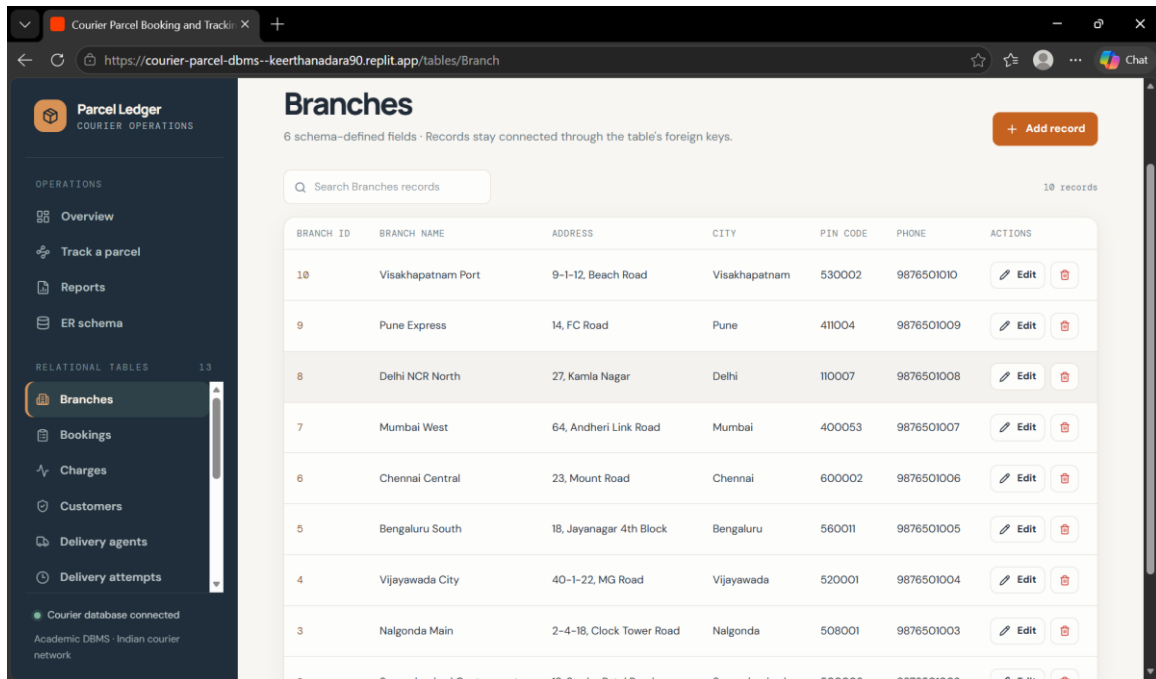
ER / RELATIONAL

Branches	Bookings	Charges	Customers
PK branch_id (Integer)	PK booking_id (Integer)	PK charge_id (Integer)	PK customer_id (Integer)
branch_name (text)	FK parcel_id (Integer)	FK booking_id (Integer)	customer_name (text)
address (text)	FK customer_id (Integer)	FK service_type_id (Integer)	phone (text)
city (text)	FK branch_id (Integer)	weight (decimal)	email (text)
pincode (pincode)	FK service_type_id (Integer)	base_charge (decimal)	address (text)
phone (phone)	booking_date (date)	additional_charge (decimal)	city (text)
	booking_status (text)	total_charge (decimal)	pincode (pincode)
	total_amount (decimal)	2 outgoing relationships	
	4 outgoing relationships		

Delivery agents	Delivery attempts	Hubs	Parcels
PK agent_id (Integer)	PK attempt_id (Integer)	PK hub_id (Integer)	PK parcel_id (Integer)
agent_name (text)	FK parcel_id (Integer)	hub_name (text)	tracking_number (text)
phone (phone)	FK agent_id (Integer)	location (text)	FK customer_id (Integer)
email (email)	attempt_date (date)	city (text)	sender_address (text)

Figure 6: ER schema page showing tables with primary and foreign keys

11.5 Table pages: view, insert, edit and delete



Parcel Ledger
COURIER OPERATIONS

OPERATIONS

- Overview
- Track a parcel
- Reports
- ER schema

RELATIONAL TABLES 13

- Branches
- Bookings
- Charges
- Customers
- Delivery agents
- Delivery attempts

Courier database connected
Academic DBMS - Indian courier network

Branches

6 schema-defined fields · Records stay connected through the table's foreign keys.

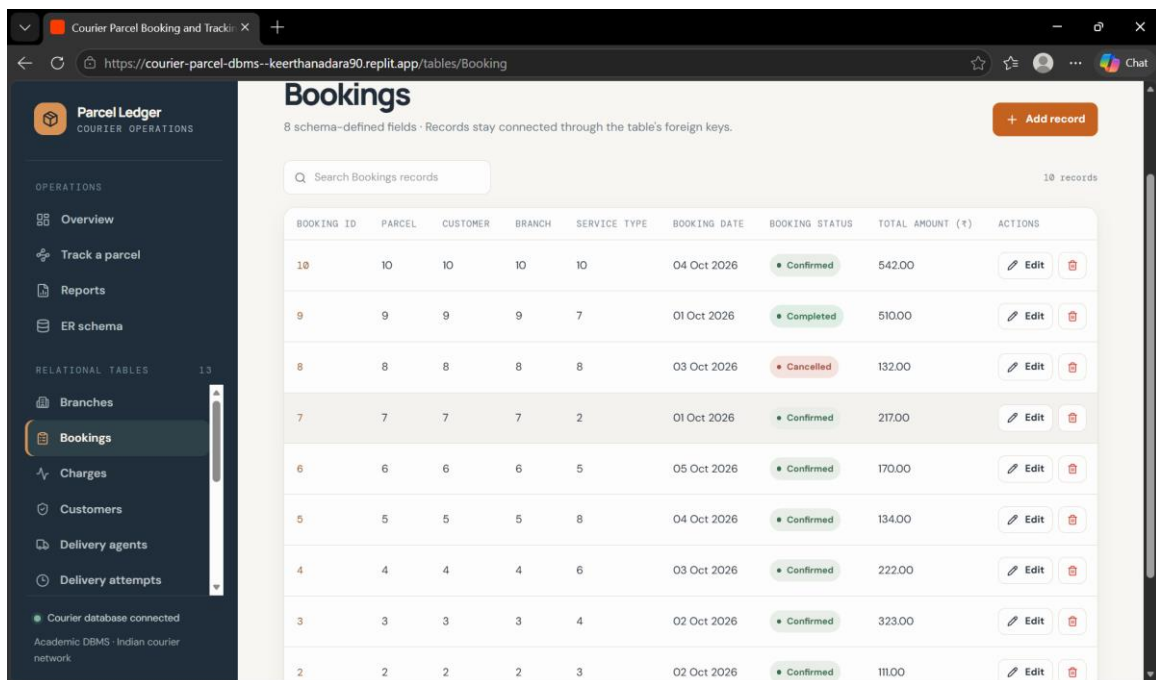
+ Add record

Search Branches records

10 records

BRANCH ID	BRANCH NAME	ADDRESS	CITY	PIN CODE	PHONE	ACTIONS
10	Visakhapatnam Port	9-1-12, Beach Road	Visakhapatnam	530002	9876501010	Edit Delete
9	Pune Express	14, FC Road	Pune	411004	9876501009	Edit Delete
8	Delhi NCR North	27, Kamla Nagar	Delhi	110007	9876501008	Edit Delete
7	Mumbai West	64, Andheri Link Road	Mumbai	400053	9876501007	Edit Delete
6	Chennai Central	23, Mount Road	Chennai	600002	9876501006	Edit Delete
5	Bengaluru South	18, Jayanagar 4th Block	Bengaluru	560011	9876501005	Edit Delete
4	Vijayawada City	40-1-22, MG Road	Vijayawada	520001	9876501004	Edit Delete
3	Nalgonda Main	2-4-18, Clock Tower Road	Nalgonda	508001	9876501003	Edit Delete

Figure 7: Branches table



Parcel Ledger
COURIER OPERATIONS

OPERATIONS

- Overview
- Track a parcel
- Reports
- ER schema

RELATIONAL TABLES 13

- Branches
- Bookings
- Charges
- Customers
- Delivery agents
- Delivery attempts

Courier database connected
Academic DBMS - Indian courier network

Bookings

8 schema-defined fields · Records stay connected through the table's foreign keys.

+ Add record

Search Bookings records

10 records

BOOKING ID	PARCEL	CUSTOMER	BRANCH	SERVICE TYPE	BOOKING DATE	BOOKING STATUS	TOTAL AMOUNT (₹)	ACTIONS
10	10	10	10	10	04 Oct 2026	Confirmed	542.00	Edit Delete
9	9	9	9	7	01 Oct 2026	Completed	510.00	Edit Delete
8	8	8	8	8	03 Oct 2026	Cancelled	132.00	Edit Delete
7	7	7	7	2	01 Oct 2026	Confirmed	217.00	Edit Delete
6	6	6	6	5	05 Oct 2026	Confirmed	170.00	Edit Delete
5	5	5	5	8	04 Oct 2026	Confirmed	134.00	Edit Delete
4	4	4	4	6	03 Oct 2026	Confirmed	222.00	Edit Delete
3	3	3	3	4	02 Oct 2026	Confirmed	323.00	Edit Delete
2	2	2	2	3	02 Oct 2026	Confirmed	111.00	Edit Delete

Figure 8: Bookings table with Edit and Delete actions

Parcel Ledger
COURIER OPERATIONS

RELATIONAL RECORDS · CHARGE_ID IS PRIMARY KEY

Charges

7 schema-defined fields · Records stay connected through the table's foreign keys.

+ Add record

Search Charges records

10 records

CHARGE ID	BOOKING	SERVICE TYPE	WEIGHT (KG)	BASE CHARGE (₹)	ADDITIONAL CHARGE (₹)	TOTAL CHARGE (₹)	ACTIONS
10	10	10	1.10	520.00	0.00	520.00	Edit
9	9	7	8.50	340.00	102.00	442.00	Edit
8	8	8	0.35	125.00	0.00	125.00	Edit
7	7	2	1.60	185.00	0.00	185.00	Edit
6	6	5	5.00	70.00	60.00	130.00	Edit
5	5	8	0.45	125.00	0.00	125.00	Edit
4	4	6	3.10	160.00	37.20	197.20	Edit

OPERATIONS

- Overview
- Track a parcel
- Reports
- ER schema

RELATIONAL TABLES

- Branches
- Bookings
- Charges
- Customers
- Delivery agents
- Delivery attempts

Courier database connected
Academic DBMS · Indian courier network

Figure 9: Charges table

Parcel Ledger
COURIER OPERATIONS

RELATIONAL RECORDS · CUSTOMER_ID IS PRIMARY KEY

Customers

7 schema-defined fields · Records stay connected through the table's foreign keys.

+ Add record

Search Customers records

10 records

CUSTOMER ID	CUSTOMER NAME	PHONE	EMAIL	ADDRESS	CITY	PIN
10	Nisha Varma	9100001010	nisha.varma@example.in	3-45, MVP Colony, Sector I	Visakhapatnam	530
9	Karthik Deshmukh	9100001009	karthik.deshmukh@example.in	Flat 8, Deccan Gymkhana	Pune	4110
8	Priya Verma	9100001008	priya.verma@example.in	A-17, Model Town, Phase II	Delhi	1100
7	Ishaan Patel	9100001007	ishaan.patel@example.in	Flat 3B, Lokhandwala Complex	Mumbai	400
6	Sana Khan	9100001006	sana.khan@example.in	17, Besant Nagar, 2nd Avenue	Chennai	600
5	Rohan Iyer	9100001005	rohan.iyer@example.in	No. 24, 5th Main, Jayanagar	Bengaluru	560
4	Meera Nair	9100001004	meera.nair@example.in	12-8-15, Governorpet	Vijayawada	520
3	Vikram Rao	9100001003	vikram.rao@example.in	6-2-140, Main Road, Ramanaguda	Nalgonda	508

OPERATIONS

- Overview
- Track a parcel
- Reports
- ER schema

RELATIONAL TABLES

- Branches
- Bookings
- Charges
- Customers
- Delivery agents
- Delivery attempts

Courier database connected
Academic DBMS · Indian courier network

Figure 10: Customers table

Parcel Ledger
COURIER OPERATIONS

RELATIONAL RECORDS · AGENT_ID IS PRIMARY KEY

Delivery agents

7 schema-defined fields · Records stay connected through the table's foreign keys.

Search Delivery agents records

10 records

AGENT ID	AGENT NAME	PHONE	EMAIL	BRANCH	VEHICLE	AGENT STATUS	ACTIONS
10	Sandeep Rao	9300003010	sandeep.rao@example.in	10	10	Active	Edit Delete
9	Rahul Jadhav	9300003009	rahul.jadhav@example.in	9	9	Active	Edit Delete
8	Pooja Singh	9300003008	pooja.singh@example.in	8	8	Active	Edit Delete
7	Imran Shaikh	9300003007	imran.shaikh@example.in	7	7	On Leave	Edit Delete
6	Divya Sundar	9300003006	divya.sundar@example.in	6	6	Active	Edit Delete
5	Manoj Shetty	9300003005	manoj.shetty@example.in	5	5	Active	Edit Delete
4	Harsha Vardhan	9300003004	harsha.vardhan@example.in	4	4	Active	Edit Delete
3	Srinivas Naidu	9300003003	srinivas.naidu@example.in	3	3	Active	Edit Delete

Parcel Ledger > Delivery agents

OPERATIONS OVERVIEW / IN OPS

RELATIONAL TABLES · 13

- Overview
- Track a parcel
- Reports
- ER schema
- Branches
- Bookings
- Charges
- Customers
- Delivery agents**
- Delivery attempts

Courier database connected
Academic DBMS · Indian courier network

Figure 11: Delivery agents table

Parcel Ledger
COURIER OPERATIONS

RELATIONAL RECORDS · ATTEMPT_ID IS PRIMARY KEY

Delivery attempts

7 schema-defined fields · Records stay connected through the table's foreign keys.

Search Delivery attempts records

10 records

ATTEMPT ID	PARCEL	DELIVERY AGENT	ATTEMPT DATE	ATTEMPT TIME	ATTEMPT STATUS	REMARKS
10	10	10	08 Oct 2026	11:03:00	In Progress	Delivery is progressing through the assigned route
9	9	9	06 Oct 2026	10:56:00	Delivered	Parcel handed to recipient with proof of delivery.
8	8	8	06 Oct 2026	09:49:00	Rescheduled	Delivery is progressing through the assigned route
7	7	7	04 Oct 2026	15:42:00	Failed	Recipient was unavailable; delivery reattempt required
6	6	6	12 Oct 2026	14:35:00	In Progress	Delivery is progressing through the assigned route
5	5	5	06 Oct 2026	13:28:00	In Progress	Delivery is progressing through the assigned route

Parcel Ledger > Delivery Attempt

OPERATIONS OVERVIEW / IN OPS

RELATIONAL TABLES · 13

- Overview
- Track a parcel
- Reports
- ER schema
- Branches
- Bookings
- Charges
- Customers
- Delivery agents
- Delivery attempts**
- Hubs
- Parcels
- Payments
- Routes
- Scan events

Courier database connected
Academic DBMS · Indian courier network

Figure 12: Delivery attempts table

Parcel Ledger > Delivery Attempt

RELATIONAL RECORDS • ATTEMPT_ID IS PRIMARY KEY

Delivery attempts

7 schema-defined fields · Records stay connected through the table's foreign keys.

Q 10 X 10 records · search results

DELIVERY AGENT	ATTEMPT DATE	ATTEMPT TIME	ATTEMPT STATUS	REMARKS	ACTIONS
10	08 Oct 2026	11:03:00	In Progress	Delivery is progressing through the assigned route.	Edit
9	06 Oct 2026	10:56:00	Delivered	Parcel handed to recipient with proof of delivery.	Edit
8	06 Oct 2026	09:49:00	Rescheduled	Delivery is progressing through the assigned route.	Edit
7	04 Oct 2026	15:42:00	Failed	Recipient was unavailable; delivery reattempt required.	Edit
6	12 Oct 2026	14:35:00	In Progress	Delivery is progressing through the assigned route.	Edit
5	06 Oct 2026	13:28:00	In Progress	Delivery is progressing through the assigned route.	Edit

Figure 13: Search feature: Delivery attempts filtered by the keyword "10"

Parcel Ledger > Payment

RELATIONAL RECORDS • PAYMENT_ID IS PRIMARY KEY

Payments

7 schema-defined fields · Records stay connected through the table's foreign keys.

Q Search Payments records X 10 records

PAYMENT ID	BOOKING	PAYMENT DATE	AMOUNT (₹)	PAYMENT METHOD	PAYMENT STATUS	TRANSACTION REFERENCE	ACTIONS
10	10	04 Oct 2026	542.00	Wallet	Paid	CP-TXN-202610-0010	Edit
9	9	01 Oct 2026	510.00	Net Banking	Paid	CP-TXN-202610-0009	Edit
8	8	03 Oct 2026	132.00	UPI	Refunded	CP-TXN-202610-0008	Edit
7	7	01 Oct 2026	217.00	Card	Failed	CP-TXN-202610-0007	Edit
6	6	05 Oct 2026	170.00	UPI	Pending	CP-TXN-202610-0006	Edit
5	5	04 Oct 2026	134.00	Cash	Paid	CP-TXN-202610-0005	Edit

Figure 14: Payments table

Parcel Ledger
COURIER OPERATIONS

OPERATIONS

- Overview
- Track a parcel
- Reports
- ER schema

RELATIONAL TABLES 13

- Delivery attempts
- Hubs
- Parcels
- Payments
- Routes**
- Scan events

Courier database connected
Academic DBMS - Indian courier network

RELATIONAL RECORDS · ROUTE_ID IS PRIMARY KEY

Routes

6 schema-defined fields · Records stay connected through the table's foreign keys.

Search Routes records

10 records

ROUTE ID	SOURCE HUB	DESTINATION HUB	ROUTE NAME	DISTANCE (KM)	ESTIMATED TIME	ACTIONS
10	10	1	MVP Colony Hub to Abids Sorting Hub	455.00	10h 83m	Edit Delete
9	9	10	Deccan Hub to MVP Colony Hub	408.00	9h 76m	Edit Delete
8	8	9	Kamla Nagar Hub to Deccan Hub	361.00	8h 69m	Edit Delete
7	7	8	Andheri Hub to Kamla Nagar Hub	314.00	7h 62m	Edit Delete
6	6	7	Teynampet Hub to Andheri Hub	267.00	6h 55m	Edit Delete
5	5	6	Jayanagar Hub to Teynampet Hub	220.00	5h 48m	Edit Delete
4	4	5	Benz Circle Hub to Jayanagar Hub	173.00	4h 41m	Edit Delete
3	3	4	Clock Tower Hub to Benz Circle Hub	126.00	3h 34m	Edit Delete

Figure 15: Routes table

Parcel Ledger
COURIER OPERATIONS

OPERATIONS

- Overview
- Track a parcel
- Reports
- ER schema

RELATIONAL TABLES 13

- Parcels
- Payments
- Routes
- Scan events**
- Service types
- Vehicles

Courier database connected
Academic DBMS - Indian courier network

RELATIONAL RECORDS · 7 schema-defined fields · Records stay connected through the table's foreign keys.

Scan events

Search Scan events records

26 records

SCAN ID	PARCEL	HUB	SCAN DATE	SCAN TIME	SCAN TYPE	REMARKS	ACTIONS
15	6	6	05 Oct 2026	08:25:00	Booked	Booking created; pickup is scheduled.	Edit Delete
26	10	2	04 Oct 2026	12:45:00	In Transit	Temperature-controlled vehicle departed.	Edit Delete
25	10	1	04 Oct 2026	10:45:00	Picked Up	Collected from sender.	Edit Delete
14	5	6	04 Oct 2026	10:20:00	Picked Up	Collected from sender.	Edit Delete
24	10	10	04 Oct 2026	08:45:00	Booked	Booking accepted at origin branch.	Edit Delete
13	5	5	04 Oct 2026	08:20:00	Booked	Booking accepted at origin branch.	Edit Delete
20	8	9	03 Oct 2026	10:35:00	Cancelled	Shipment cancelled at sender request.	Edit Delete
12	4	5	03 Oct 2026	10:15:00	At Hub	Arrived at destination sorting hub.	Edit Delete
19	8	8	03 Oct 2026	08:35:00	Booked	Booking accepted at origin branch.	Edit Delete

Figure 16: Scan events table

RELATIONAL RECORDS · SERVICE_TYPE_ID IS PRIMARY KEY

Service types

5 schema-defined fields · Records stay connected through the table's foreign keys.

+ Add record

Search Service types records

10 records

SERVICE TYPE ID	SERVICE NAME	DESCRIPTION	DELIVERY DAYS	BASE CHARGE (₹)	ACTION
10	Cold Chain	Temperature-controlled delivery for eligible parcels.	2	520.00	Edit
9	International Priority	Priority export courier service with end-to-end tracking.	5	890.00	Edit
8	Document Secure	Tracked and tamper-evident document delivery.	2	125.00	Edit
7	Bulk Freight	Consolidated freight service for larger shipments.	6	340.00	Edit
6	Fragile Care	Handle-with-care service for delicate consignments.	3	160.00	Edit
5	Economy Surface	Budget-friendly non-urgent surface delivery.	7	70.00	Edit
4	Same Day Metro	Same-day delivery within supported metropolitan zones.	1	275.00	Edit

Figure 17: Service types table

RELATIONAL RECORDS · VEHICLE_ID IS PRIMARY KEY

Vehicles

6 schema-defined fields · Records stay connected through the table's foreign keys.

+ Add record

Search Vehicles records

10 records

VEHICLE ID	VEHICLE NUMBER	VEHICLE TYPE	CAPACITY (KG)	BRANCH	VEHICLE STATUS	ACTIONS
10	AP3IPK1010	Mini van	800.00	10	Available	Edit Delete
9	MH12NJ9009	Electric cargo bike	55.00	9	On Route	Edit Delete
8	DLOIMH8008	Mini truck	1800.00	8	Available	Edit Delete
7	MH02LG7007	Cargo van	1400.00	7	In Service	Edit Delete
6	TN09KF6006	Mini van	950.00	6	Available	Edit Delete
5	KA03JE5005	Electric cargo bike	45.00	5	On Route	Edit Delete
4	AP16HD4004	Mini truck	2200.00	4	Available	Edit Delete
3	TS05GC3003	Cargo van	1250.00	3	Available	Edit Delete
2	TS10FB2002	Motorcycle	25.00	2	On Route	Edit Delete

Figure 18: Vehicles table

11.6 Insert and delete screens

[INSERT SCREENSHOTS: (a) "Add record" form before insert, (b) table after insert, (c) delete confirmation, (d) table after delete. The course instructions ask for before and after screenshots of insert, delete and view.]

12. Implementation Details

12.1 Technology stack

Layer	Technology
Database	SQLite relational database with 13 tables, 1 view and foreign key enforcement
Application / API layer	Courier API that runs SQL against the connected tables and returns JSON to the interface
User interface	Web pages (HTML, CSS, JavaScript) deployed on Replit
Source control	GitHub repository (link in Appendix)

[Replace "Application / API layer" and "User interface" with the exact framework you used, e.g. Node.js + Express, React, Flask.]

12.2 Database connectivity

The application opens the SQLite database when it starts, enables foreign key checking (PRAGMA foreign_keys = ON), and initialises the schema. The home page confirms this with the message "Database initialized successfully - 13 tables created and connected". All data shown in the interface is read from, or written to, these tables, so the UI always reflects the database state.

12.3 Key SQL operations behind the interface

View all records of a table:

```
SELECT * FROM Booking ORDER BY booking_id DESC;
```

Insert a record from the "Add record" form (values are bound as parameters to prevent SQL injection):

```
INSERT INTO Customer (customer_name, phone, email, address, city, state, postcode)
VALUES (?, ?, ?, ?, ?, ?, ?);
```

Delete a record from the Actions column:

```
DELETE FROM Customer WHERE customer_id = ?;
```

Track a parcel by tracking number:

```
SELECT p.tracking_no, p.status, s.scan_type, s.scan_time, b.branch_name, s.remarks
FROM Parcel p
JOIN Scan_Event s ON p.parcel_id = s.parcel_id
LEFT JOIN Branch b ON s.branch_id = b.branch_id
WHERE p.tracking_no = ?
ORDER BY s.scan_time;
```

12.4 Integrity rules implemented

- Primary keys on all tables; foreign keys for every relationship.
- UNIQUE: customer phone and email, tracking number, vehicle registration number, booking.parcel_id, charge.booking_id.

- CHECK: weight, dimensions, capacity and charges must be positive; additional charge cannot be negative.
- DEFAULT: new parcels start with status BOOKED; payment status defaults to SUCCESS.

13. Testing (Test Cases and Results)

13.1 Database integrity tests

The following SQL statements were executed to check that the constraints reject invalid data and accept valid data.

No.	Test case	Expected result	Actual result	Status
T1	Duplicate customer phone	Rejected (UNIQUE)	Rejected: UNIQUE failed: Customer.phone	Pass
T2	Negative parcel weight	Rejected (CHECK)	Rejected: CHECK failed: weight > 0	Pass
T3	Duplicate tracking number	Rejected (UNIQUE)	Rejected: UNIQUE failed: Parcel.tracking_no	Pass
T4	Booking with unknown customer	Rejected (FOREIGN KEY)	Rejected: FOREIGN KEY failed	Pass
T5	Second booking for same parcel	Rejected (UNIQUE parcel_id)	Rejected: UNIQUE failed: Booking.parcel_id	Pass
T6	Delete branch that has a delivery agent	Rejected (FOREIGN KEY)	Rejected: FOREIGN KEY failed	Pass
T7	Insert valid parcel with default status	Accepted; status defaults to BOOKED	Accepted; status = BOOKED	Pass
T8	Delete the new unreferenced parcel	Accepted	Accepted	Pass

13.2 User interface tests

No.	Test case	Expected result	Status
U1	Open the Overview page	Dashboard loads with counts for all entities and a "database connected" message	Pass
U2	Open each of the 13 table pages	All records are listed with their foreign key values	Pass
U3	Search "10" on the Delivery attempts page	Only matching records are listed (10 records)	Pass
U4	Track parcel CPIN2026100010	Shipment record, current hub and scan history are displayed	Pass
U5	Open the Reports page and run "All customers"	SQL text and sorted customer list are displayed	Pass
U6	Open the ER schema page	All 13 tables with PK/FK fields are displayed	Pass
U7	Add a record using "Add record"	New record appears in the table and the count increases	Pass
U8	Delete a record using the delete button	Record is removed and the count decreases	Pass

14. Conclusion and Future Enhancements

14.1 Conclusion

The project successfully designed and implemented a relational database for courier parcel booking and tracking. The ER diagram with 13 entities was converted into a 3NF schema, implemented in SQL with integrity constraints, loaded with sample data and queried using joins, aggregates and a view. The web interface lets users view, add, edit and delete records, track a parcel through its scan history and run predefined reports. Testing confirmed that the constraints reject invalid data. The system shows how a DBMS can bring booking, routing, delivery and payment data into one consistent, queryable store.

14.2 Future enhancements

- User authentication with roles (customer, branch staff, administrator).
- Customer self-service portal and SMS / e-mail notifications on every scan event.
- Online payment gateway integration and automatic invoice generation.
- Automatic charge calculation by a trigger from service type and weight.
- Real-time GPS tracking of vehicles and route optimisation.
- Migration to a client-server DBMS such as MySQL or PostgreSQL for multi-user production use; indexes and stored procedures for performance.
- Analytics dashboards for delivery performance and revenue.

15. References

- Silberschatz, A., Korth, H. F., and Sudarshan, S., Database System Concepts, McGraw-Hill Education.
- Elmasri, R., and Navathe, S. B., Fundamentals of Database Systems, Pearson.
- Ramakrishnan, R., and Gehrke, J., Database Management Systems, McGraw-Hill.
- SQLite Documentation: <https://www.sqlite.org/docs.html>
- W3Schools SQL Tutorial: <https://www.w3schools.com/sql/>
- DBMS Course Project Instructions, Presentation-I/II/III guidelines.

16. Appendix: GitHub Repository Link

The complete project (report, ER diagram, SQL script, source code and screenshots) is available at:

<https://github.com/darakeerthana25/courier-parcel-booking-tracking>

Live application: <https://courier-parcel-dbms--keerthanadara90.replit.app>